🎓 Aula 3 – Conhecendo o Jest (Testar como um Dev Profissional)

Primeiros Passos:

Primeiro, a visão geral
Quando alguém diz que vai testar uma aplicação em TypeScript com Jest, normalmente está falando deste conjunto:
TypeScript
É a linguagem do projeto. Ele adiciona tipagem ao JavaScript.
Jest
É a ferramenta que executa os testes e mostra se passaram ou falharam.
ts-jest
É a ponte entre TypeScript e Jest. Faz o Jest entender arquivos .ts.
@types/jest
Entrega ao TypeScript o conhecimento sobre funções como ***test***, ***expect***, ***describe***.

“Agora complicou 😱, será?”

Analogia simples

Pensa assim:

- **TypeScript** é o idioma em que o sistema foi escrito
- **Jest** é o avaliador
- **ts-jest** é o tradutor
- **@types/jest** é o manual que ensina as palavras do avaliador ao TypeScript

Sem isso, o projeto até pode existir, mas a conversa entre as peças fica quebrada.

Estrutura de um projeto simples

Uma estrutura inicial comum seria assim:

```
meu-projeto/
    src/
        sum.ts
    tests/
        sum.test.ts
  package.json
  tsconfig.json
  jest.config.ts
```

O papel de cada arquivo

src/
Aqui fica o código da aplicação.
tests/
Aqui ficam os testes. Em alguns projetos os testes ficam junto do arquivo original. Exemplo: sum.spec.ts dentro de src/.
package.json
Controla dependências e scripts do projeto.
tsconfig.json
Configura o TypeScript.
jest.config.ts
Configura o Jest.

1. Setup do zero

Agora vamos simular um projeto real.

1. Criar o projeto Node

```
npm init -y
```

Isso cria o package.json.

2. Instalar dependências de desenvolvimento

npm install -D typescript jest ts-jest @types/jest

Aqui está o motivo de cada uma:

typescript para escrever e compilar TS
jest para rodar testes
ts-jest para integrar TS com Jest
@types/jest para o TypeScript reconhecer a API do Jest

3. Gerar o tsconfig

npx **tsc** –init

Isso cria o tsconfig.json.

4. Gerar configuração do Jest

npx ts-jest config:init

Isso cria um arquivo de configuração do ***Jest***, normalmente algo assim:

```
import type { Config } from 'jest';

const config: Config = {
  preset: 'ts-jest',
  testEnvironment: 'node',
};

export default config;
```

O que essa configuração quer dizer: 🤯

preset: 'ts-jest'
Diz ao Jest que ele vai trabalhar com TypeScript usando o ts-jest.

testEnvironment: 'node'
Diz que os testes vão rodar em ambiente Node.js, e não navegador.

Isso é o mais comum em APIs, serviços, regras de negócio e back end.

Ajustando o package.json

Você normalmente adiciona um script assim:

JSON

```
{
  "scripts": {
    "test": "jest"
  }
}
```

Então para rodar os testes:

npm test

```
PS C:\Users\comercial3\OneDrive - SPS Power\Área de Trabalho\Jest-aula> npm test

> projeto-banco@1.0.0 test
> jest

 PASS  tests/soma.test.ts
  Função soma
    √ deve somar dois números corretamente (3 ms)

Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:   0 total
Time:        0.605 s, estimated 1 s
Ran all test suites.
```

UC11 – Executar testes e Melhorias em Aplicações

Primeiro exemplo funcionando

Arquivo `src/sum.ts`

TypeScript

```typescript
export function sum(a: number, b: number): number {
  return a + b;
}
```

Arquivo `tests/sum.test.ts`

TypeScript

```typescript
import { sum } from '../src/sum';

test('deve somar dois números corretamente', () => {
  expect(sum(2, 3)).toBe(5);
});
```

Obrigado!

Prof. Éder Silva

Senac